

Teste de Software

Roteiro *Sprint 2* - Geração do `confest.py` via GitHub Copilot Chat

Continuando: a sequência abaixo substitui o arquivo `test_ia_gerado.py` construído a partir da Q8, o Copilot Chat do GitHub agora precisa produzir o arquivo `confest.py` a partir da observação crítica que o você registrou no Passo 5 do *Sprint 1* sobre a omissão da fixture de isolamento. O insumo do *Sprint 2* é a saída do *Sprint 1*.

Passo 1. Abra o arquivo Word produzido por você no *Sprint 1*, localize a tabela de critérios técnicos do **Passo 5** e copie o texto completo da coluna **Observação** da linha *Fixture de isolamento (setup/teardown)*. Este texto é o insumo do prompt do Passo 4, assim como a Q8 foi insumo do prompt do *Sprint 1*.

Passo 2. Confirme que o arquivo `test_ia_gerado.py` produzido no *Sprint 1* está salvo dentro da pasta `internet_banking_tests`. Sem este artefato, o *Sprint 2* não pode ser executado, a fixture a ser gerada existe para corrigir o comportamento empírico observado nesse arquivo.

Passo 3. Abra o servidor Flask no VSCode dentro do ambiente virtual `venv` ativado. No primeiro terminal:

```
python app.py
```

Passo 4. Abra o GitHub Copilot Chat no VSCode (Cmd+Shift+I no macOS / Ctrl+Alt+I no Windows). No editor, crie um novo arquivo chamado `conftest.py`. **COLE O PROMPT ABAIXO** no campo de chat do GitHub Copilot, substituindo `[SUA_OBSERVACAO_SPRINT1]` pelo texto copiado no Passo 1, e pressione Enter:

Gere o conteúdo de um arquivo `conftest.py` com fixture pytest para resolver a seguinte omissão técnica identificada na minha análise crítica dos testes gerados por IA no Sprint 1 da disciplina de Teste de Software:

`[SUA_OBSERVACAO_SPRINT1]`

Contexto técnico: os testes estão no arquivo `test_ia_gerado.py` e fazem requisições HTTP reais para um servidor Flask local (`http://127.0.0.1:5000`) sobre o arquivo `app.py` de um sistema de internet banking. O banco SQLite `banking.db` possui as tabelas `contas` e `transferencias`. O estado inicial a ser restaurado antes de cada teste é: `id=1` (Alice) saldo 1000.00; `id=2` (Bob) saldo 500.00; `id=3` (Carlos) saldo 0.00; tabela `transferencias` completamente limpa.

Requisitos obrigatórios:

- Use o decorador `@pytest.fixture(autouse=True)` para aplicação automática a todos os testes do arquivo `test_ia_gerado.py`.
- Use a biblioteca padrão `sqlite3`.
- Execute o reset ANTES de cada teste, com `yield` ao final da fixture.
- Feche a conexão com o banco antes do `yield`.

Gere apenas o código Python. Sem explicações adicionais.

Com o servidor Flask já rodando no Terminal 1 (Passo 3), **não feche esse terminal**. Abra um segundo terminal no VSCode (Ctrl+Shift+`), ative a mesma `venv` e execute os testes do *Sprint 1* agora com a fixture do Copilot ativa:

```
# Windows
.\venv\Scripts\activate

# Linux / macOS
source venv/bin/activate

# Execução dos testes do Sprint 1 com a fixture autouse ativa
pytest test_ia_gerado.py -v
```

Investigação empírica: no *Sprint 1* a *primeira* execução do pytest passava em todos os doze testes e a *segunda* falhava em `test_transferencia_saldo_igual_valor` com HTTP 422 - saldo insuficiente. Com a fixture `autouse` do `conftest.py` gerado pelo Copilot agora aplicada, execute o pytest duas vezes consecutivas no Terminal 2 e registre se o comportamento se manteve ou se a violação da característica de independência de testes (ISTQB CTFL v4.0, Cap. 6) foi corrigida. Este registro é o insumo do Passo 3 do Sprint 2 - classificação das falhas por categoria.

Roteiro *Sprint 2* - Comparação, cobertura e reflexão

Passo 5. Compare quantitativamente os dois *sprints* preenchendo a tabela abaixo. A coluna *Sprint 1* vem do registro que você fez no Passo 5 do roteiro da semana anterior; a coluna *Sprint 2* vem do resultado da verificação empírica do Passo 4 deste *sprint*, executada após a aplicação do `conftest.py`:

Métrica	<i>Sprint 1</i> (sem conftest)	<i>Sprint 2</i> (com conftest)
Testes que passaram na 1ª execução do pytest		
Testes que passaram na 2ª execução do pytest		
Nome do teste que falhava entre execuções		
Categoria da falha diagnosticada (A / B / C / D)		

Passo 6. A taxonomia de falhas possíveis na execução de uma suíte de testes em ambiente de integração compreende quatro categorias:

(A) **Erro de teste** - o teste está mal escrito; o sistema está correto.

(B) **Erro de API** - endpoint errado ou payload incorreto.

(C) **Defeito real** - o sistema retornou valor incorreto para uma entrada válida.

(D) **Erro de isolamento** - passou na 1ª execução e falhou na 2ª por dependência de estado mutável.

Na sua tabela do Passo 5, identifique qual foi a *única* categoria empiricamente manifestada no Sprint 1 e que foi *eliminada* pela fixture autouse do confest.py no Sprint 2. Em seguida, em um parágrafo registrado em seu arquivo Word, analise por que as outras três categorias **não apareceram** nesta suíte específica gerada pela IA a partir da sua Q8 -atribuindo a omissão ao escopo da especificação humana (você não pediu cenários adversariais), ao limite da ferramenta (a IA não inferiu cenários fora do que foi solicitado), ou a ambos. Sustente sua análise em **COUTINHO; NASCIMENTO (2025)** com indicação de página. Observe a implicação: *uma suíte que passa não é, sozinha, evidência de que o sistema está correto* - apenas de que ele responde corretamente ao que foi pedido.

Passo 7. Com o servidor Flask ainda rodando no Terminal 1, execute novamente o pytest no Terminal 2, agora com relatório de cobertura de código:

```
pytest test_ia_gerado.py -v --cov=. --cov-report=term-missing
```

No relatório que o pytest exibirá no terminal, localize a coluna **Missing** - ela lista, por arquivo, as linhas do app.py que *nenhum* teste do test_ia_gerado.py executou. Registre o relatório completo (copie e cole no seu arquivo Word) e o percentual de cobertura total exibido na coluna **Cover**.

Passo 8. Para *cada* linha do app.py registrada na coluna Missing do relatório do Passo 7, responda em seu arquivo Word:

a) qual cenário de negócio do sistema de internet banking essa linha de código implementa? Identifique o endpoint (/saldo, /transferencia ou /extrato) e descreva, em uma frase, a regra de negócio que está implementada naquela linha.

b) por que a IA não gerou um teste que ativasse essa linha? Sustente a hipótese em referência do material da disciplina - a omissão tem origem na sua especificação da Q8 (você não pediu), no limite da ferramenta (ela não inferiu) ou em ambos?

Reflexão do *Sprint 2* - registre em seu arquivo

Responda com suas próprias palavras. Não existe resposta certa. O que importa é que o texto revele o seu raciocínio real durante a execução.

1. Como mudou o número de testes que passaram entre o Sprint 1 (sem conftest) e o Sprint 2 (com conftest)? O que essa diferença mostra empiricamente sobre o papel da fixture de isolamento na característica de independência de testes (ISTQB CTFL v4.0, Capítulo 6)?

2. Das quatro categorias possíveis de falha (A / B / C / D), apenas uma se manifestou empiricamente nas suas execuções e foi eliminada pela fixture. Por que as outras três não apareceram nesta suíte gerada pela IA a partir da sua Q8 — limite da especificação, limite da ferramenta, ou ambos?

3. Quais linhas do app.py ficaram com cobertura zero? Qual risco concreto isso representa para o internet banking se o sistema fosse colocado em produção exatamente como está, com a suíte de testes que a IA gerou?

4. Com base em **BSTQB (2023, p. 52)**, o que a cobertura de linhas garante — e o que ela **não** garante sobre a qualidade do código testado? Cite a página.

Poste no ambiente virtual.ifro.edu.br todas as respostas em formato Word requeridas neste *sprint*. – Vale até 5 pontos.